perm_pateda: Solving permutation-based optimization problems with Estimation of Distribution Algorithms implemented in Python

Julen Benagues Ekhine Irurozki Leticia Hernando Josu Ceberio
Roberto Santana
Department of Computer Science and Artificial Intelligence
University of the Basque Country (UPV/EHU)
leticia.hernando,josu.ceberio,roberto.santana{@ehu.eus}

2026

Abstract

This document describes `perm_pateda`, a Python library of estimation of distribution algorithms (EDAs) for optimization problems whose solutions are *permutations*. `perm_pateda` is a specialized extension of `pateda`[1]: it contributes everything that is specific to the permutation space — distances, central-permutation (consensus) estimators, permutation probability models, their learning and sampling methods, permutation seeding, benchmark problems and instance readers — while reusing `pateda` for everything common to all EDAs (the core EDA executor, component interfaces, selection, replacement, statistics, stopping conditions and visualization). The library covers the five distance-based EDAs of the `perm_mateda` Matlab toolbox (Mallows and generalized Mallows models under the Kendall's-τ, Cayley and Ulam distances) and extends them with a substantially wider catalogue of permutation models: kernels of Mallows models under the Hamming distance, edge and node histogram models, the Plackett–Luce model and mixtures of Plackett–Luce models, doubly stochastic matrix models, univariate/tree/Markov-chain models over bijective codings of permutations (Lehmer codes, Fisher–Yates draws, insertion vectors), and random-key EDAs that search the unit hypercube with continuous models. On the problem side, `perm_pateda` implements the four classical permutation problems (TSP, PFSP, LOP, QAP) together with readers for the standard benchmark libraries (TSPLIB, Taillard, LOLIB, QAPLIB), and three graph problems reformulated under the "permutation picture" (maximum independent set, maximum cut, minimum vertex cover). A decomposition-based multi-objective framework (MEDA/D) instantiated with nine different permutation models is also provided. Whenever a functionality is inherited unchanged from `pateda`, this guide points to the corresponding section of the `pateda` user guide instead of repeating its description.

Contents

[1] `https://github.com/rsantana-isg/pateda`

1 Introduction

Estimation of distribution algorithms (EDAs) [29, 31, 38, 40] replace the variation operators of genetic algorithms by the explicit estimation and sampling of a probabilistic model. At each generation a model is learned from a set of selected solutions, and new solutions are obtained by sampling that model. The general description of the EDA paradigm, of the probabilistic graphical models used in the discrete and continuous cases, and of the modular architecture on which this library is built, is given in the `pateda` user guide and is not repeated here.

Many important combinatorial optimization problems, however, are naturally defined over the *symmetric group* $\mathcal{S}_n$: the travelling salesman problem (TSP), the permutation flowshop scheduling problem (PFSP), the linear ordering problem (LOP), and the quadratic assignment problem (QAP), among many others. A solution to such a problem is a permutation σ of n items, i.e. a bijection $\sigma : \{1, \ldots, n\} \to \{1, \ldots, n\}$. The number of solutions is $n!$, and, crucially, the set of permutations is *not* a Cartesian product of independent variable domains. A model that treats the n positions of a permutation as n independent integer variables — the natural choice for a discrete EDA — assigns positive probability to vectors that are not permutations at all, so that sampling produces infeasible individuals that must be repaired, distorting the distribution the model was supposed to represent [5].

`perm_pateda` therefore replaces the model family, not the algorithm. The EDA loop, the selection and replacement operators, the statistics and the stopping criteria are those of `pateda`; what changes is (i) how a population of permutations is represented, (ii) which probability distributions over $\mathcal{S}_n$ are estimated from the selected set, and (iii) how new permutations are drawn from them — always guaranteeing feasibility by construction. Three complementary strategies are implemented:

- **Distributions defined directly on $\mathcal{S}_n$.** Distance-based exponential models (Mallows, generalized Mallows, kernels of Mallows models), multistage models (Plackett–Luce and its mixtures), first- and second-order statistics of the population (node and edge histogram models), and doubly stochastic matrix models. Sampling from these models always returns a valid permutation.

- **Bijective codings of $\mathcal{S}_n$.** Lehmer codes, Fisher–Yates draw sequences and insertion vectors map $\mathcal{S}_n$ one-to-one onto a product of bounded integer domains $\{0\} \times \{0, 1\} \times \cdots \times \{0, \ldots, n-1\}$. In that space *every* vector decodes to a valid permutation, so the whole battery of discrete EDA models of `pateda` (univariate, tree-structured, Markov chains) becomes directly applicable.

- **Continuous relaxations.** Random keys [3] encode a permutation as a real vector in $[0, 1]^n$ whose sorting order defines the permutation. Any continuous EDA of `pateda` (Gaussian univariate, full multivariate Gaussian, vine copulas) can then be used to search the permutation space.

The main design goals of `perm_pateda` follow those of `pateda`:

- *Plug-and-play convenience:* classes such as `MallowsKendallEDA`, `EHMEDA`, `PlackettLuceEDA` or `DSMASEDA` provide a one-liner interface to a complete permutation EDA.

- *Flexible component assembly:* every learning and sampling method is an independent object with the `pateda` component interface, so it can be plugged into a generic `EDA` through `EDAComponents`.

- *Extensibility:* adding a new permutation model requires only a learner (selected population $\to$ model dictionary) and a sampler (model dictionary $\to$ new permutations).

- *Single- and multi-objective support:* a decomposition-based multi-objective framework (MEDA/D) is provided and can be instantiated with any of nine permutation models.
- *Reproducibility:* instance readers for the standard benchmark libraries and statistical comparison utilities (Friedman, pairwise Wilcoxon, critical-difference diagrams) are included.

The remainder of this guide is organized as follows. Section 2 describes installation and the package structure. Section 3 explains how a permutation EDA is defined and executed. Section 4 surveys the plug-and-play algorithm classes. Section 5 details the probability models over permutations, their learning and their sampling. Section 6 documents the EDA components, indicating which are inherited from `pateda`. Section 7 lists the permutation problems and instance readers. Section 8 covers the multi-objective framework. Section 9 describes model inspection and the statistical comparison utilities. Section 10 discusses the injection of a-priori knowledge. Section 11 relates the library to the `perm_mateda` toolbox.

2 Installation and Package Structure

2.1 Installation

`perm_pateda` depends on `pateda`. Since `pateda` is an in-development package, install it first from the local checkout and then install `perm_pateda`:

```
pip3 install -e packages/pateda
pip3 install -e packages/perm_pateda
```

For the development tooling (`pytest`, `ruff`, `black`, `mypy`):

```
pip3 install -e "packages/perm_pateda[dev]"
```

Required dependencies are `numpy`, `scipy` and `pateda`. The mixture of Plackett–Luce models additionally uses `scikit-learn` (k-means in the spectral initialization), and the statistical utilities use `pandas` and `matplotlib`. Python ≥ 3.9 is required.

2.2 Package Structure

The top-level `perm_pateda` namespace exposes the algorithm classes, the distances, the consensus estimators, the random-key conversion helpers, the benchmark readers and the statistical utilities:

```
from perm_pateda import (
    # distance-based EDAs
    MallowsKendallEDA, MallowsCayleyEDA, MallowsUlamEDA,
    GMallowsKendallEDA, GMallowsCayleyEDA, HammingKMMEDA,
    # histogram, multistage and matrix models
    EHMEDA, NHMEDA, PlackettLuceEDA, PlackettLuceMixtureEDA,
    DSMPSEDA, DSMASEDA,
    # random-key EDAs
    RKGaussianUMDAEDA, RKGaussianFullEDA, RKCopulaVinesEDA,
    # distances and consensus
    kendall_distance, cayley_distance, ulam_distance,
        hamming_distance,
    find_consensus_borda, find_consensus_median,
    # instance readers
    parse_lolib, parse_taillard_pfsp, parse_qaplib, parse_tsplib,
    # random-key utilities
    permutation_to_random_keys, random_keys_to_permutation,
    random_keys_to_ranks, rescale_random_keys,
```

```
    # statistical comparison
    summary_table, friedman_test, wilcoxon_pairwise,
    critical_difference_plot,
)
```

The EDAs built on bijective codings (`LehmerUmdaEDA`, `LehmerTreeEDA`, `FisherYatesUmdaEDA`, `FisherYatesTreeEDA`, `InsertionVectorUmdaEDA`, `InsertionVectorMarkovEDA`) are imported from `perm_pateda.algorithms`. The internal module tree is:

Module	Contents
`distances.py`	Kendall's-τ, Cayley, Ulam and Hamming distances; inversion vector $\mathbf{V}$, cycle-based decomposition vector $\mathbf{X}$, derangement numbers
`consensus.py`	Central-permutation estimators (Borda, set median, best) and their dispatcher; permutation composition and inversion
`random_keys.py`	Conversion between permutations, random keys and ranks; rank-based rescaling
`representations/`	Bijective codings: `LehmerRepresentation`, `FisherYatesRepresentation`, `InsertionVectorRepresentation`
`learning/`	All permutation model learners
`sampling/`	All permutation model samplers
`seeding/`	`PermutationInit` (uniform random permutations)
`algorithms/`	Plug-and-play wrappers (`permutation.py`, `random_keys.py`) and `pateda` component adapters (`base.py`)
`functions/`	Permutation problems: TSP, PFSP, LOP, QAP, MIS, MaxCut, MVC, with random-instance generators
`multiobjective/`	Scalarization, Pareto dominance and archive, the MEDA/D family of decomposition-based multi-objective permutation EDAs
`utils/`	Benchmark instance parsers and statistical comparison tools

Note that there are no `selection/`, `replacement/`, `stop_conditions/`, `statistics/` or `knowledge_extraction/` modules: those components are representation agnostic and are imported directly from `pateda`.

2.3 Running the Examples

The `examples/` directory contains runnable scripts that show the two ways of assembling a permutation EDA — through the generic `pateda` executor and through the plug-and-play wrappers:

```
python3 examples/ehm_tsp_example.py
python3 examples/mallows_tsp_example.py
```

The test suite (`tests/`) exercises the Mallows model under the Cayley distance and the generalized Mallows models, checking the decomposition vectors, the learn→sample round trip and convergence on small instances:

```
pytest tests/
```

3 Defining and Executing a Permutation EDA

3.1 General Description

A permutation EDA follows the generic EDA schema of `pateda` (Algorithm 1): an initial population of permutations is generated and evaluated; at each generation a subset of the population is selected, a probability model over $\mathcal{S}_n$ is estimated from it, and new permutations are sampled from the model to form the next population. The only structural differences with respect to the discrete EDA of `pateda` are that the initialization returns permutations, and that the learning/sampling pair operates on $\mathcal{S}_n$.

Algorithm 1 Estimation of distribution algorithm for permutation problems

1: $t \leftarrow 0$
2: Generate D_0: N permutations drawn uniformly at random from $\mathcal{S}_n$
3: Evaluate D_0
4: **repeat**
5: $\quad D_t^{Se} \leftarrow$ Select $M \leq N$ permutations from D_t
6: $\quad p_t(\sigma) \leftarrow$ Estimate a probability distribution over $\mathcal{S}_n$ from D_t^{Se}
7: $\quad D_{t+1} \leftarrow$ Sample N permutations from $p_t(\sigma)$ and evaluate them
8: $\quad t \leftarrow t + 1$
9: **until** stopping criterion is met

Two execution paths are available. The *component path* builds a `pateda.core.eda.EDA` object out of an `EDAComponents` dataclass in which the learning and sampling slots are filled with `perm_pateda` objects; this gives access to the whole `pateda` machinery (arbitrary selection and replacement operators, stopping conditions, statistics tracking, local optimization, caching). The *wrapper path* uses the plug-and-play classes of Section 4, which implement a self-contained loop.

3.2 The Permutation EDA Wrappers

All plug-and-play classes derive from an internal base class `_PermEDA` (`algorithms/permutation.py`) that implements the loop of Algorithm 1 with truncation selection and optional elitism. A self-contained loop is used — instead of delegating to `pateda.core.eda.EDA` — because several permutation learners and samplers expose only `__call__` (rather than `.learn()` / `.sample()`), and because the samplers need an explicit `sample_size` argument. The base class bridges both calling conventions transparently.

3.2.1 Input Parameters

All permutation algorithm classes share the following constructor parameters:

Parameter	Description
`n_vars`	Permutation length n; items are $0, \ldots, n-1$
`fitness_func`	Callable mapping a 1-D integer array (a permutation) to a scalar; higher is better
`pop_size`	Population size N (default: 100)
`n_gen`	Number of generations (default: 50)
`selection_ratio`	Truncation fraction M/N (default: 0.5)
`elitism`	Preserve the best permutation found so far (default: True)
`random_seed`	Seed of the `numpy` random generator (default: None)

Model-specific parameters are added by individual classes and are listed in Table 1.

3.2.2 Output: `Statistics` and `Cache`

`run()` returns the pair `(stats, cache)` of `pateda` objects. `Statistics` accumulates, per generation, the best, mean and worst fitness, the population diversity measures, and the best individual found overall (`stats.best_fitness_overall`, `stats.best_individual`). Their fields and the `Cache` contents are documented in the `pateda` user guide; the permutation wrappers populate them with exactly the same semantics.

3.3 Assembling a Permutation EDA from Components

Because the learners and samplers implement the `pateda` `LearningMethod` / `SamplingMethod` interfaces (directly, or through the adapters in `algorithms/base.py`), a permutation EDA can be assembled component by component exactly like any other `pateda` algorithm:

```python
import numpy as np
from pateda.core.eda import EDA
from pateda.core.components import EDAComponents
from pateda.selection import TruncationSelection
from pateda.replacement import ElitistReplacement
from pateda.stop_conditions import MaxGenerations

from perm_pateda.seeding import PermutationInit
from perm_pateda.learning.histogram import LearnEHM
from perm_pateda.sampling.histogram import SampleEHM
from perm_pateda.functions import create_random_tsp

n_cities, pop_size, n_gen = 15, 80, 40
tsp = create_random_tsp(n_cities, seed=42)

components = EDAComponents(
    seeding=PermutationInit(),
    selection=TruncationSelection(ratio=0.5),
    learning=LearnEHM(),
    sampling=SampleEHM(),
    replacement=ElitistReplacement(n_elite=int(pop_size * 0.1)),
    stop_condition=MaxGenerations(n_gen),
)
components.learning_params = {"symmetric": True, "beta_ratio": 0.01}
components.sampling_params = {"sample_size": pop_size}

eda = EDA(n_vars=n_cities,
          cardinality=np.full(n_cities, n_cities, dtype=int),
          fitness_func=tsp, pop_size=pop_size,
          components=components, random_seed=42)
stats, cache = eda.run()
print("Best tour length:", -stats.best_fitness_overall)
```

Two remarks. The `learning_params` and `sampling_params` dictionaries carry the model-specific arguments that the generic executor does not know about — in particular `sample_size`, which the permutation samplers require. And the `cardinality` argument, which is essential for discrete EDAs, carries no information here: it is passed for interface compatibility only, and the plug-and-play wrappers fill it with the placeholder `np.arange(n_vars)`.

3.4 Example: Mallows EDA on the Linear Ordering Problem

The equivalent program using a plug-and-play wrapper is a single object construction:

```
import numpy as np
from perm_pateda import MallowsKendallEDA
from perm_pateda.functions import create_random_lop

lop = create_random_lop(15, seed=0)

alg = MallowsKendallEDA(
    n_vars=15,
    fitness_func=lop,          # LOP.__call__: permutation -> scalar
    pop_size=100,
    n_gen=50,
    selection_ratio=0.3,
    random_seed=0,
)
stats, _ = alg.run(verbose=True)
print("Best fitness:    ", stats.best_fitness_overall)
print("Best permutation:", stats.best_individual)
```

3.5 Fitness Convention

As in `pateda`, **the fitness function is always maximized**. The problem classes of Section 7 follow this convention internally: `TSP.__call__` returns the *negated* tour length, `QAP.__call__` the negated assignment cost, `PFSP.__call__` the negated makespan or flowtime, and `MVC.__call__` the negated size of the vertex cover, while `LOP`, `MIS` and `MaxCut` return directly maximized quantities. Each class additionally provides an `evaluate_*` method returning the natural (positive, non-negated) objective value for reporting:

```
tour_length  = tsp.evaluate_distance(perm)    # positive
qap_cost     = qap.evaluate_cost(perm)        # positive
makespan     = pfsp.evaluate_makespan(perm)   # positive
lop_value    = lop.evaluate_objective(perm)
```

3.6 Indexing Convention

Permutations are internally 0-indexed arrays containing each of $0, \ldots, n-1$ exactly once. For compatibility with instance files and with the Matlab toolbox, the learners, samplers and problem classes detect 1-indexed input (by testing whether the minimum entry equals one) and convert it internally. Users writing new components should produce 0-indexed permutations.

4 Plug-and-Play Algorithm Classes

Table 1 summarizes the permutation algorithm classes. Every class exposes the constructor parameters of Section 3 plus the extra parameters listed in the last column, and a `run(verbose=False)` method returning `(stats, cache)`.

Table 1: Plug-and-play permutation algorithm classes in `perm_pateda`.

Class	Identifier	Model	Extra parameters
Distance-based exponential models			

Class	Identifier	Model	Extra parameters
`MallowsKendallEDA`	MEDA_K	Mallows model, Kendall's-τ distance [9]	—
`MallowsCayleyEDA`	MEDA_C	Mallows model, Cayley distance [25]	—
`MallowsUlamEDA`	MEDA_U	Mallows model, Ulam distance, MCMC sampling [24]	`burn_in`, `step_size`
`GMallowsKendallEDA`	GMEDA_K	Generalized Mallows, Kendall's-τ [6]	—
`GMallowsCayleyEDA`	GMEDA_C	Generalized Mallows, Cayley [7]	—
`HammingKMMEDA`	KMM_H	Kernels of Mallows models under the Hamming distance [1]	`expected_dist_start`, `expected_dist_end`, `gamma`
Histogram, multistage and matrix models			
`EHMEDA`	EHM-EDA	Edge histogram model [47]	—
`NHMEDA`	NHM-EDA	Node histogram model [48]	—
`PlackettLuceEDA`	PL-EDA	Plackett–Luce model estimated with the MM algorithm [10]	—
`PlackettLuceMixtureEDA`	MPL-EDA	Mixture of K Plackett–Luce models, spectral EM [39]	`n_components`
`DSMPSEDA`	DSM-PS	Doubly stochastic matrix, probabilistic sampling [45]	`alpha`
`DSMASEDA`	DSM-AS	Doubly stochastic matrix, algebraic sampling [45]	`alpha`
Models over bijective codings			
`LehmerUmdaEDA`	L-UMDA	Univariate marginals over Lehmer codes [42]	`laplace_smoothing`
`LehmerTreeEDA`	L-Tree	Chow–Liu tree over Lehmer codes [13]	`laplace_smoothing`, `root`
`FisherYatesUmdaEDA`	FY-UMDA	Univariate marginals over Fisher–Yates draws [17]	`laplace_smoothing`
`FisherYatesTreeEDA`	FY-Tree	Chow–Liu tree over Fisher–Yates draws	`laplace_smoothing`, `root`
`InsertionVectorUmdaEDA`	IV-UMDA	Univariate marginals over insertion vectors	`laplace_smoothing`
`InsertionVectorMarkovEDA`	IV-MC	First-order Markov chain over insertion vectors	`laplace_smoothing`
Random-key (continuous relaxation) EDAs			
`RKGaussianUMDAEDA`	RK-UMDA	Univariate Gaussian on random keys [2]	`diminishing`, `cooling`, `initial_sigma`, `min_sigma`
`RKGaussianFullEDA`	RK-Full	Full multivariate Gaussian on random keys	idem
`RKCopulaVinesEDA`	RK-Vine	Vine-copula model on random keys	idem + `truncation_level`

4.1 Quick Comparison Example

```
import numpy as np
from perm_pateda import (MallowsKendallEDA, GMallowsCayleyEDA,
                         EHMEDA, PlackettLuceEDA, DSMASEDA)
from perm_pateda.functions import create_random_qap

n = 20
qap = create_random_qap(n, seed=1)

```

```
for AlgClass in [MallowsKendallEDA, GMallowsCayleyEDA,
                 EHMEDA, PlackettLuceEDA, DSMASEDA]:
    alg = AlgClass(n_vars=n, fitness_func=qap,
                   pop_size=200, n_gen=100, random_seed=1)
    stats, _ = alg.run()
    print(f"{AlgClass.__name__:22s} cost = "
          f"{-stats.best_fitness_overall:.0f}")
```

5 Probability Models over Permutations

5.1 Permutation Representation and Notation

Permutations are denoted by σ or π; π^{-1} is the inverse of π; the composition of σ and π is $\sigma\pi$, defined by $\sigma\pi(i) = \sigma(\pi(i))$; $e = 1\,2\,3\cdots n$ is the identity permutation. A distance $d(\sigma, \pi)$ measures the disagreement between the two permutations and, for convenience, $d(\sigma, e)$ is written $d(\sigma)$. All the distances considered here are *right invariant*,

$$d(\sigma, \pi) = d(\sigma\pi^{-1}, e) = d(\sigma\pi^{-1}), \tag{1}$$

a property that is repeatedly exploited by the learning and sampling algorithms.

A population of N permutations of length n is stored as a 2-D `numpy` array of shape (N, n) and integer `dtype`, in the same way that `pateda` stores a discrete population.

5.2 Distances on Permutations

The module `distances.py` implements four distances together with the auxiliary decompositions required by the models.

Kendall's-τ distance. $d_\tau(\sigma, \pi)$ counts the number of pairwise disagreements between σ and π:

$$d_\tau(\sigma, \pi) = |\{(i, j) : i < j,\ (\sigma(i) < \sigma(j) \wedge \pi(i) > \pi(j)) \\ \vee(\pi(i) < \pi(j) \wedge \sigma(i) > \sigma(j))\ \}|.$$

It is the best choice when permutations are interpreted as *rankings* (as in recommender systems), and it is equivalent to the minimum number of adjacent swaps needed to convert σ^{-1} into π^{-1}. The Kendall's-τ distance decomposes into the *inversion vector* $\mathbf{V}(\sigma) = (V_1(\sigma), \ldots, V_{n-1}(\sigma))$,

$$V_j(\sigma) = \sum_{i=j+1}^{n} \mathbf{1}_{[\sigma(i)<\sigma(j)]}, \tag{2}$$

so that $V_j(\sigma)$ is the number of items with index larger than j whose value is smaller than $\sigma(j)$, with $0 \le V_j(\sigma) \le n - j$. For example, if $\sigma = 2\,1\,3\,6\,4\,5$, then $\mathbf{V}(\sigma) = (1\,0\,0\,2\,0)$ and $d_\tau(\sigma) = \sum_{j=1}^{n-1} V_j(\sigma) = 3$. The conversion between $\mathbf{V}(\sigma)$ and σ takes $O(n^2)$. In `perm_pateda` the inversion vector is computed by `_v_vector` and the distance by `kendall_distance`.

Cayley distance. $d_c(\sigma, \pi)$ counts the minimum number of swaps (not necessarily adjacent) needed to transform σ into π. It is intimately related to the cycles of σ — ordered sets $\{i_1, \ldots, i_s\}$ such that $\sigma(i_1) = i_2$, $\sigma(i_2) = i_3$, …, $\sigma(i_s) = i_1$ — and when the reference permutation is the identity, $d_c(\sigma)$ equals n minus the number of cycles of σ. Its decomposition vector $\mathbf{X}(\sigma) = (X_1(\sigma), \ldots, X_{n-1}(\sigma))$ is

$$X_j(\sigma) = \begin{cases} 0 & \text{if } j \text{ is the largest item in its cycle in } \sigma, \\ 1 & \text{otherwise,} \end{cases} \tag{3}$$

computable in $O(n)$. For $\sigma = 2\,1\,3\,6\,4\,5$, whose cycle notation is $(2\,1)(3)(4\,5\,6)$, one gets $\mathbf{X}(\sigma) = 1\,0\,0\,1\,1$ and $d_c(\sigma) = 3$. The functions `_x_vector_cycles` and `_generate_perm_from_x` implement the two directions of this conversion, the latter being the core of the Cayley sampler.

Ulam distance. $d_u(\sigma, \pi)$ is n minus the length of the longest common subsequence of σ and π; when the reference is the identity, $d_u(\sigma)$ is n minus the length of the longest increasing subsequence (LIS). The classical illustration [16] is a shelf of books ordered as σ that must be reordered as π using the minimum number of *delete-and-insert* movements; that minimum is exactly $d_u(\sigma, \pi)$. For $\sigma = 2\,1\,3\,6\,4\,5\,7$ the LIS has length 5, hence $d_u(\sigma) = 2$. The implementation (`ulam_distance`) computes the LIS in $O(n \log n)$ with the patience-sorting algorithm. *No distance decomposition vector is known for the Ulam distance*, which is why the generalized Mallows model cannot be extended to it [24].

Hamming distance. $d_H(\sigma, \pi) = |\{i : \sigma(i) \neq \pi(i)\}|$ counts the positions at which the two permutations disagree. It is the natural distance when the absolute position of each item is what matters, as in the QAP, and it underlies the kernels of Mallows models of Section 5.7. The number of permutations at Hamming distance k from a fixed permutation is $S(n, k) = \binom{n}{k} D_k$, where D_k is the number of derangements of k elements, computed by `compute_derangements` through the recurrence $D_k = (k-1)(D_{k-1} + D_{k-2})$. Note that d_H can never equal one.

Choice of distance. There is no distance that is best in general. As reported by **(author?)** [8], some distances behave well for some problems (meaning that they perform well for most instances of the problem) and other distances for other problems. Although an exhaustive correspondence between distances and problems is not possible, as a general rule an EDA under a particular distance will perform well on a problem when that metric is adequate to measure the discrepancies between permutations in that domain. The correspondence suggested by the experimental study of the `perm_mateda` toolbox associates the Kendall's-τ distance with the PFSP, the Cayley distance with the QAP, and the Ulam distance with the LOP.

5.3 The Mallows Model

The Mallows model [33] is a distance-based exponential probability model over permutation spaces. Given a distance over permutations, it is defined by two parameters: the central permutation σ_0 and the spread parameter θ. The probability of every permutation σ is

$$P(\sigma) = \psi(\theta)^{-1} \exp\big(-\theta\, d(\sigma, \sigma_0)\big), \tag{4}$$

where $\psi(\theta)$ is the normalization constant. When $\theta > 0$, σ_0 is the mode of the distribution, and the probability of the remaining $n! - 1$ permutations decreases exponentially with their distance to σ_0: the closer a permutation is to σ_0, the higher its probability. When $\theta = 0$ the model reduces to the uniform distribution over $\mathcal{S}_n$, and as θ grows the distribution concentrates around σ_0. The Mallows model is thus the exponential-family analogue, on $\mathcal{S}_n$, of the Gaussian distribution on $\mathbb{R}^n$, with σ_0 playing the role of the mean and θ^{-1} that of the variance.

5.4 The Generalized Mallows Model

The generalized Mallows (GM) model [19] is an extension of the Mallows model which is also exponential and unimodal. Instead of a single spread parameter θ, the GM model uses an $(n-1)$-dimensional spread parameter $\boldsymbol{\theta} = (\theta_1, \ldots, \theta_{n-1})$, where each θ_j affects a particular position of the permutation. This allows the distribution to put more emphasis on the consensus of certain positions while keeping more uncertainty in others.

Not every distance that can be used in the Mallows model can be used in the GM model. The GM model requires the distance to decompose as a sum of $n-1$ terms,

$$d(\sigma, \sigma_0) = \sum_{j=1}^{n-1} S_j(\sigma\sigma_0^{-1}), \tag{5}$$

where the vector $\mathbf{S}(\sigma\sigma_0^{-1}) = (S_1(\sigma\sigma_0^{-1}), \ldots, S_{n-1}(\sigma\sigma_0^{-1}))$ is the distance decomposition vector. The GM model is then

$$P(\sigma) = \psi(\boldsymbol{\theta})^{-1} \exp\left(\sum_{j=1}^{n-1} -\theta_j\, S_j(\sigma\sigma_0^{-1})\right). \tag{6}$$

The decomposition vector is $\mathbf{V}(\cdot)$ of Eq. (2) for the Kendall's-τ distance and $\mathbf{X}(\cdot)$ of Eq. (3) for the Cayley distance. In both cases the interpretation of θ_j is the same: the larger θ_j, the larger the probability that position j agrees with the consensus. Since no decomposition vector is known for the Ulam distance, `perm_pateda` provides five distance-based EDAs rather than six: MEDA_K, MEDA_C, MEDA_U, GMEDA_K and GMEDA_C.

5.5 Learning the Mallows and Generalized Mallows Models

Learning the parameters of a Mallows or GM model from a sample of permutations is either known or conjectured to be NP-hard for all the distances considered here. Within an EDA, however, the model that most accurately represents the selected solutions need not be learned at every generation: approximate models can even be beneficial, since they help to explore other areas of the search space, and they are computationally much cheaper. The learning step is therefore split into two stages [23]:

1. Approximate the central permutation σ_0.
2. Obtain the maximum-likelihood estimate (MLE) of the spread parameter(s) $\boldsymbol{\theta}$ for the given distance.

Stage 1: central permutation. Three estimators are implemented in `consensus.py` and selected through the `consensus_method` argument of every Mallows/GM learner (`"borda"`, the default; `"setmedian"`; `"best"`):

- **Borda** [14] (`find_consensus_borda`): selects as central permutation the result of sorting the items according to their average position across all the input permutations. When the permutations come from a Mallows model under the Kendall's-τ distance, the Borda permutation is an asymptotically optimal estimator of the central permutation [20]. Although it can be used with any distance, it is recommended exclusively for the Kendall's-τ distance.
- **SetMedianPermutation** (`find_consensus_median`): selects the individual of the sample that minimizes the sum of distances to the rest, under the distance of the model. It is $O(M^2)$ in the number M of selected individuals and is the recommended default for the Cayley and Ulam distances.
- **BestPermutation** (`find_consensus_best`): chooses the permutation with the best objective value. This estimator requires the fitness values, which the learners receive as `selected_fitness`.

The dispatcher `get_consensus(method, population, fitness, distance_func)` implements the selection logic and raises an informative error when the information required by an estimator (fitness for `"best"`, a distance for `"setmedian"`) has not been supplied.

Stage 2: spread parameters. Once σ_0 has been obtained, the spread parameter(s) are estimated by equating the derivative of the log-likelihood to zero. The resulting expressions differ for each distance, and in most cases a numerical method is used to approximate the solution; exact expressions can be found in **(author?)** [23, 25, 34]. `perm_pateda` uses:

- `LearnMallowsKendall`: the observed mean inversion vector $\overline{\mathbf{V}}$ is matched against its expectation under the model, and a bounded scalar minimization (`scipy.optimize.fminbound`) over $\theta \in (0, \texttt{upper\_theta}]$ returns the MLE. The normalization constants $\psi_j(\theta)$ are computed in closed form.
- `LearnGeneralizedMallowsKendall`: one θ_j per position is obtained by solving the corresponding univariate equation, giving the $(n-1)$-dimensional $\boldsymbol{\theta}$ together with the per-position probability matrix of the V_j values used by the sampler.
- `LearnMallowsCayley` and `LearnGeneralizedMallowsCayley`: the mean of the cycle-based decomposition $\overline{\mathbf{X}}$ is used, and Newton's method (`scipy.optimize.newton`, with the analytic derivative) solves the likelihood equation; the resulting ψ constants yield the vector of probabilities $P(X_j = 1)$ consumed by the sampler.
- `LearnMallowsUlam`: since the Ulam distance does not factorize, the learner matches the *expected* distance under the model with the mean observed distance to σ_0. This requires the number N_d of permutations at each Ulam distance d, which is enumerated exactly for $n \leq 8$ and approximated by a Gaussian profile centred at $n - 2\sqrt{n}$ for larger n.

All learners return a plain Python dictionary, which is the model container used throughout `perm_pateda`; for instance `{"consensus": `σ_0`, "theta": `θ`, "model_type": "mallows_kendall", ...}`. This makes models trivially serializable and inspectable (Section 9).

5.6 Sampling the Mallows and Generalized Mallows Models

Sampling means generating permutations from the model obtained in the learning stage, and the algorithm depends on the distance.

Mallows and GM models under the Kendall's-τ and Cayley distances *factorize*, which leads to efficient sampling algorithms. Given the per-position distributions obtained in the learning step, the sampler first draws a decomposition vector ($\mathbf{V}$ for Kendall's-τ, $\mathbf{X}$ for Cayley) coordinate by coordinate, and then converts that vector into a permutation, which is finally composed with σ_0. This is the *multistage sampler* of **(author?)** [23], implemented by `SampleMallowsKendall`, `SampleMallowsCayley`, `SampleGeneralizedMallowsKendall` and `SampleGeneralizedMallowsCayley`. Its complexity is dominated by the vector-to-permutation conversion, $O(n^2)$ for Kendall's-τ and $O(n)$ per step for Cayley.

For the Ulam distance, sampling a permutation at a prescribed distance is equivalent to sampling random standard Young tableaux, which is the *distances sampler* of **(author?)** [23]. `perm_pateda` takes a simpler route: `SampleMallowsUlam` draws from Eq. (4) with a Metropolis–Hastings Markov chain whose proposal is a random transposition, accepting a move with probability $\min\{1, \exp(-\theta\,\Delta d_u)\}$. The chain is run for `burn_in` steps (default 1000 in `MallowsUlamEDA`) and one sample is collected every `step_size` steps (default 100). When $\theta > 10$ the distribution is numerically degenerate and the sampler returns `sample_size` copies of σ_0, avoiding a needless chain.

```
import numpy as np
from perm_pateda.learning.mallows import LearnMallowsCayley
from perm_pateda.sampling.mallows import SampleMallowsCayley

n, m = 10, 50
rng  = np.random.default_rng(0)
```

```
pop  = np.vstack([rng.permutation(n) for _ in range(m)])

learner = LearnMallowsCayley()
model = learner(generation=0, n_vars=n, cardinality=np.arange(n),
                selected_pop=pop, selected_fitness=np.zeros(m),
                consensus_method="setmedian")
print(model["model_type"], model["theta"])

sampler = SampleMallowsCayley()
new_pop = sampler(n_vars=n, model=model, cardinality=np.arange(n),
                  population=pop, fitness=np.zeros(m),
                  sample_size=100, rng=rng)
```

5.7 Kernels of Mallows Models under the Hamming Distance

Instead of estimating a single Mallows model centred on a consensus permutation, a *kernel* model [1] places one Mallows kernel on *each* selected solution and samples from the resulting mixture,

$$P(\sigma) = \frac{1}{M} \sum_{i=1}^{M} \psi(\theta)^{-1} \exp\big(-\theta\, d_H(\sigma, \sigma_i)\big), \tag{7}$$

where $\sigma_1, \ldots, \sigma_M$ are the selected permutations. This preserves the diversity of the selected set, which a single unimodal model destroys, while still concentrating the probability mass around good solutions.

`LearnHammingKMM` stores the selected population as the set of centres and computes θ from a *scheduled* expected distance rather than by maximum likelihood. The expected Hamming distance decreases exponentially along the run, from E_0 (default $n/2$, parameter `expected_dist_start`) to $E_{\max}$ (default 0.25, parameter `expected_dist_end`):

$$E(t) = E_{\max} + \delta(p)\,(E_0 - E_{\max}), \qquad \delta(p) = \frac{e^{-\gamma p} - 1}{e^{-\gamma} - 1}, \qquad p = \frac{t}{t_{\max}}, \tag{8}$$

with γ (default 5.14) controlling the decay rate. The value of θ matching a target expected distance is obtained by bisection on

$$\mathbb{E}[d_H] = \frac{\sum_k k\, S(n,k)\, e^{-\theta k}}{\sum_k S(n,k)\, e^{-\theta k}}, \qquad S(n,k) = \binom{n}{k} D_k, \tag{9}$$

computed in log-space for numerical stability and excluding the impossible value $k = 1$. `SampleHammingKMM` then implements the exact three-step sampler: choose a centre σ_0 uniformly at random among the stored centres, draw a distance k with probability proportional to $S(n,k)e^{-\theta k}$, and build a permutation at Hamming distance exactly k from σ_0 by choosing k positions and deranging them. This schedule gives the algorithm an explicit exploration-to-exploitation transition without any adaptive parameter.

5.8 Histogram Models: EHM and NHM

Histogram models summarize the selected population with a matrix of first- or second-order statistics. They were introduced for permutation problems by **(author?)** [47] and compared in **(author?)** [48].

Node histogram model (NHM). The NHM records how often each item appears at each position,

$$\mathrm{NHM}[i,j] \;\propto\; \big|\{\sigma \in D^{Se} : \sigma(i) = j\}\big| + \varepsilon, \qquad \varepsilon = \beta_{\text{ratio}} \cdot M, \tag{10}$$

each row being normalized to a probability distribution. The bias ε, controlled by `beta_ratio`, plays the role of a mutation operator: it prevents zero probabilities and keeps every item reachable at every position. `SampleNHM` constructs a permutation position by position, drawing the item at position i from the (renormalized) row i restricted to the items not yet used, falling back to the uniform distribution over the remaining items when the restricted row has zero mass.

Edge histogram model (EHM). The EHM records how often item b immediately follows item a,

$$\mathrm{EHM}[a,b] \;=\; \big|\{\sigma \in D^{Se} : \exists\, i,\ \sigma(i)=a,\ \sigma(i+1)=b\}\big| \;+\; \varepsilon, \tag{11}$$

with $\varepsilon = \beta_{\text{ratio}}\,\frac{M}{n-1}$ in the asymmetric case and $\varepsilon = \beta_{\text{ratio}}\,\frac{2M}{n-1}$ in the symmetric case (`symmetric=True`, the default), where the matrix is replaced by $\mathrm{EHM} + \mathrm{EHM}^{\top}$. The symmetric variant is the appropriate one for undirected problems such as the symmetric TSP, in which a tour and its reverse are equivalent, while the asymmetric variant should be used when the direction of a transition matters. `SampleEHM` starts from a randomly chosen item and extends the permutation greedily-at-random, at each step drawing the next item from the row of the current item restricted to the unused items. The EHM is a natural model for problems whose objective is defined on *adjacency* (TSP, PFSP), whereas the NHM suits problems defined on *absolute positions* (QAP, LOP).

5.9 The Plackett–Luce Model

The Plackett–Luce (PL) model [32, 41] is a multistage model in which the permutation is generated by sampling without replacement proportionally to a vector of item weights $\mathbf{w} = (w_1, \ldots, w_n)$, $w_i > 0$:

$$P(\sigma) = \prod_{i=1}^{n} \frac{w_{\sigma(i)}}{\sum_{j=i}^{n} w_{\sigma(j)}}. \tag{12}$$

The parameter w_i measures the propensity of item i to be ranked early. Unlike the Mallows model, the PL model is not defined through a distance, and it can represent distributions that are not unimodal in the metric sense; it was introduced in the EDA context by **(author?)** [10].

`LearnPlackettLuce` computes the MLE of $\mathbf{w}$ with the minorization–maximization (MM) algorithm of **(author?)** [22]. Writing W_i for the number of "wins" of item i (the number of stages in which i is chosen, i.e. the number of times i appears in a position other than the last), the MM update is

$$w_i \;\leftarrow\; W_i \Big/ \sum_{k=1}^{M} \sum_{p=1}^{\mathrm{pos}_k(i)} \Big(\sum_{q \ge p} w_{\sigma_k(q)} \Big)^{-1}, \tag{13}$$

followed by normalization. The implementation computes the inner double sum in $O(n)$ per ranking using suffix sums and cumulative sums, so one MM iteration costs $O(Mn)$; iterations stop after `max_iter` (default 100) or when the L_1 change of $\mathbf{w}$ falls below `tol` (default 10^{-6}).

`SamplePlackettLuce` exploits the *Gumbel-max* trick: adding i.i.d. Gumbel noise to the log-weights and sorting the perturbed values produces exactly a draw from Eq. (12). Sampling a whole population is thus a single vectorized `argsort` of an (N, n) array, i.e. $O(Nn \log n)$, instead of N sequential loops of n normalizations.

5.10 Mixtures of Plackett–Luce Models

A single PL model has one mode; when the selected population contains several distinct groups of good solutions — a common situation in multimodal permutation problems — a *mixture* of K PL components,

$$P(\sigma) = \sum_{c=1}^{K} \beta_c\, P(\sigma \mid \mathbf{w}^{(c)}), \qquad \sum_{c} \beta_c = 1, \tag{14}$$

is a far more faithful model. `LearnPlackettLuceMixture` implements the spectral-EM algorithm (EM-LSR) of **(author?)** [39], in two phases:

1. **Spectral initialization.** Every ranking is embedded as a binary pairwise-comparison vector of length $n(n-1)/2$, whose entry (i, j) is 1 if i precedes j. A truncated SVD of the resulting $(M, n(n-1)/2)$ matrix is computed, the number of retained dimensions $\hat{r}$ is chosen adaptively from the gaps between singular values, and k-means clusters the projected rankings into K groups. A least-squares estimator then converts the empirical pairwise-comparison probabilities of each cluster into an initial parameter vector.

2. **EM refinement with weighted LSR.** The E-step computes the posterior responsibility of each component for each ranking, using log-sum-exp for stability; the M-step re-estimates each component with *weighted Luce spectral ranking* [36], which is an exact MLE step rather than a surrogate, obtained as the stationary distribution of a Markov chain built from the weighted pairwise statistics.

The defaults used inside an EDA are deliberately loose (`max_em_iter=10`, `tol_em`=10^{-4}, `max_iter_lsr=20`), since a rough model estimated cheaply at every generation is preferable to an exact one estimated slowly. `SamplePlackettLuceMixture` draws the component index from $\boldsymbol{\beta}$ and then applies the Gumbel-max trick with the corresponding weight vector, again fully vectorized. The number of components is exposed as `n_components` in `PlackettLuceMixtureEDA`.

5.11 Doubly Stochastic Matrix Models

A doubly stochastic matrix (DSM) $D \in [0, 1]^{n \times n}$ — all rows and all columns summing to one — is, by the Birkhoff–von Neumann theorem, a convex combination of permutation matrices, and is therefore a natural parametric family of distributions over $\mathcal{S}_n$ [45].

`LearnDSM` implements the smoothed learning scheme: the selected permutations are converted to permutation matrices $P_1, \ldots, P_M$, averaged, and mixed with the uniform DSM U (all entries $1/n$),

$$D = \frac{1-\alpha}{M} \sum_{i=1}^{M} P_i + \alpha\, U, \tag{15}$$

where the smoothing factor $\alpha \in (0, 1]$ prevents zero entries — which would make some assignments unreachable — and defaults to $1/n^2$. Equation (15) is the exact analogue, in permutation space, of the Laplace-smoothed marginals used by the discrete EDAs of `pateda`.

Two samplers are provided:

DSM-PS (probabilistic sampling) builds one permutation iteratively: a still-unassigned row of D is chosen uniformly at random, the column is drawn from the corresponding row probabilities renormalized over the unassigned columns, and the matched row and column are removed. The probability of sampling σ is $\Pr(\sigma \mid D) = \prod_i d_{i,\sigma(i)} / \mathrm{Perm}(D)$, where $\mathrm{Perm}(D)$ is the permanent of D. Cost: $O(n^2)$ per permutation.

DSM-AS (algebraic sampling) draws a uniform random vector v, computes Dv, and reads the permutation off the ranking relationship between v and Dv. It is also $O(n^2)$ per permutation, but the cost is a matrix–vector product, so it is fully vectorizable over the whole population and, in practice, considerably faster thanks to optimized BLAS routines.

5.12 Models over Bijective Codings of $\mathcal{S}_n$

The modules `representations/` exploit a different idea: rather than defining a distribution on $\mathcal{S}_n$, map $\mathcal{S}_n$ bijectively onto a product space of bounded integers, in which every vector is a valid code, and then use a *standard discrete* EDA model. The abstract interface

`PermutationRepresentation` requires three methods: `encode(perm)`, `decode(code)` and `get_domain(n)` (the inclusive maximum value allowed at each position). All three implementations are vectorized over a population.

`LehmerRepresentation` The Lehmer code (or factoradic representation) [30, 42]. In the right (default) variant, $c_i = |\{j > i : \sigma(j) < \sigma(i)\}|$, so $c_i \in \{0, \ldots, n-1-i\}$ and $c_{n-1} = 0$; the left variant counts inversions to the left, giving $c_i \in \{0, \ldots, i\}$. The right Lehmer code is precisely the inversion vector $\mathbf{V}$ of Eq. (2), so a univariate model over Lehmer codes is closely related to a generalized Mallows model under the Kendall's-τ distance centred on the identity.

`FisherYatesRepresentation` The sequence of draws performed by the Fisher–Yates shuffle [18, 17]: at step i, $c_i \in \{0, \ldots, n-1-i\}$ is the offset of the element swapped into position i. Decoding replays the shuffle.

`InsertionVectorRepresentation` The insertion vector, obtained as the left Lehmer code of the *inverse* permutation, with $c_i \in \{0, \ldots, i\}$: c_i is the position at which item i is inserted into the partial permutation of items $0, \ldots, i-1$. It is the coding underlying insertion-based neighbourhoods, which are the natural move operator for the LOP and the PFSP.

Three families of models are learned over these codings, all of them handling the fact that the domain size varies with the position:

- `LearnUMDA` / `SampleLehmerUMDA`, `SampleFisherYatesUMDA`, `SampleInsertionVectorUMDA`: independent per-position marginals with Laplace smoothing (`laplace_alpha`, default 0.01 in the wrappers). This is the UMDA [38] transported to permutation space.
- `LearnTree` / `SampleLehmerTree`, `SampleFisherYatesTree`: a Chow–Liu tree [13] built from the pairwise mutual information between code positions, oriented from a user-selected `root`, with one conditional probability table per edge. This is the Tree-EDA of `pateda` applied to a bijective coding, and it can capture dependencies between positions that the univariate model misses.
- `LearnInsertionVectorChain` / `SampleInsertionVectorChain`: a first-order Markov chain over the insertion vector, $P(c) = P(c_0) \prod_{i \geq 1} P(c_i \mid c_{i-1})$, with smoothed conditional tables of shape $(i, i+1)$.

Because *every* code decodes to a permutation, no repair operator is ever needed, and the learned model can be inspected with the same tools as a discrete `pateda` model.

5.13 Random-Key EDAs

Random keys [3] encode a permutation as a real vector $x \in [0,1]^n$; the permutation is recovered as the ordering induced by sorting x. Since every real vector decodes to a valid permutation, any *continuous* EDA can be used to search $\mathcal{S}_n$. The module `random_keys.py` provides the conversions:

Function	Description
`random_keys_to_permutation(x)`	Stable `argsort` of the keys; works on a single vector or a whole population
`permutation_to_random_keys(p)`	Assigns evenly spaced keys in $[0,1]$ consistent with the given order, with optional jitter
`random_keys_to_ranks(x)`	1-based ranks of the keys
`rescale_random_keys(x)`	Rank-based rescaling $x_i \leftarrow (\text{rank}_i - 1)/(n-1)$

The three algorithm classes `RKGaussianUMDAEDA`, `RKGaussianFullEDA` and `RKCopulaVinesEDA` implement the RK-EDA of **(author?)** [2] with the continuous learners and samplers of `pateda` (univariate Gaussian, full multivariate Gaussian, and vine copulas respectively). The two mechanisms that make RK-EDA effective are both exposed as constructor flags:

Diminishing variance / rescaling (`diminishing=True`) The selected keys are rank-rescaled to evenly spaced values in $[0, 1]$ before the model is learned, and the sampled keys are rescaled again. This removes the arbitrary scale of the keys and keeps the mapping to permutations well conditioned.

Cooling (`cooling=True`) The standard deviation used for sampling is not the one estimated from the data but a scheduled value

$$\sigma_g = \max\left(\sigma_0 \cdot \max\left(1 - \frac{g}{G}, 0\right), \sigma_{\min}\right), \qquad \sigma_0 = \frac{1}{\pi \log_{10} n}, \tag{16}$$

which decreases linearly with the generation g. The correlation structure learned by the model is preserved: for a full-covariance model the covariance matrix is converted to a correlation matrix and rescaled to the target variance. For the vine-copula variant, which has no explicit variance parameter, Gaussian noise of standard deviation σ_g is added to the sampled keys instead.

5.14 Fourier-Based Models (experimental)

The modules `multiobjective/meda_d_f.py`, `multiobjective/fourier_moead.py` and `multiobjective/moead` implement algorithms based on the Fourier decomposition of permutation functions [12, 27], in which a pseudo-Boolean-like spectral expansion of the objective over $\mathcal{S}_n$ is truncated at a small order and used either as a surrogate for pre-screening candidates (`FourierMOEAD`) or as a probability model to be combined across objectives (`MEDA_D_F`). These three modules import a `perm_pateda.fourier` package (and the auxiliary modules `perm_pateda.distributions` and `perm_pateda.utils.permutations`) that are *not part of the current distribution*: they come from the separate `permutation_fourier` project. Consequently they are not re-exported by `perm_pateda.multiobjective.__init__` and are not importable out of the box. They are documented here for completeness and should be regarded as experimental until that dependency is vendored or declared.

5.15 Summary of the Model Catalogue

Table 2 lists every learning/sampling pair provided by `perm_pateda`, with the module in which it is implemented.

Each learner exposes the signature `learn(generation, n_vars, cardinality, population, fitness, **params)` (or the equivalent `__call__` with `selected_pop` / `selected_fitness`) and returns a model dictionary; each sampler exposes `sample(n_vars, model, cardinality, population, fitness, sample_size, rng)` and returns an `(sample_size, n_vars)` integer array of valid permutations. Implementing a new permutation model amounts to writing this pair.

6 EDA Components

Apart from learning and sampling, all the components of an EDA are representation agnostic and are inherited unchanged from `pateda`. This section states which ones apply to permutations and documents the single component that `perm_pateda` contributes.

Model	Learner	Sampler	Module
Mallows, Kendall's-τ	`LearnMallowsKendall`	`SampleMallowsKendall`	`mallows`
Mallows, Cayley	`LearnMallowsCayley`	`SampleMallowsCayley`	`mallows`
Mallows, Ulam	`LearnMallowsUlam`	`SampleMallowsUlam`	`mallows`
GM, Kendall's-τ	`LearnGeneralizedMallowsKendall`	`SampleGeneralizedMallowsKendall`	`mallows`
GM, Cayley	`LearnGeneralizedMallowsCayley`	`SampleGeneralizedMallowsCayley`	`mallows`
Mallows kernels, Hamming	`LearnHammingKMM`	`SampleHammingKMM`	`hamming_`
Edge histogram	`LearnEHM`	`SampleEHM`	`histogra`
Node histogram	`LearnNHM`	`SampleNHM`	`histogra`
Plackett–Luce	`LearnPlackettLuce`	`SamplePlackettLuce`	`plackett`
Mixture of PL	`LearnPlackettLuceMixture`	`SamplePlackettLuceMixture`	`mixture_`
Doubly stochastic matrix	`LearnDSM`	`SampleDSMPS`, `SampleDSMAS`	`dsm`
Lehmer, univariate	`LearnLehmerUMDA`	`SampleLehmerUMDA`	`lehmer`
Lehmer, tree	`LearnLehmerTree`	`SampleLehmerTree`	`lehmer`
Fisher–Yates, univariate	`LearnFisherYatesUMDA`	`SampleFisherYatesUMDA`	`fisher_ya`
Fisher–Yates, tree	`LearnFisherYatesTree`	`SampleFisherYatesTree`	`fisher_ya`
Insertion vector, univariate	`LearnInsertionVectorUMDA`	`SampleInsertionVectorUMDA`	`vinsertio`
Insertion vector, Markov chain	`LearnInsertionVectorChain`	`SampleInsertionVectorChain`	`vinsertio`

Table 2: Learning and sampling methods available in `perm_pateda`. Module names are relative to `perm_pateda.learning` and `perm_pateda.sampling`. Random-key EDAs reuse the continuous learners and samplers of `pateda`.

6.1 Seeding Methods

`PermutationInit` (`seeding/permutation_init.py`) is the permutation counterpart of `RandomInit`: it generates `pop_size` permutations of `range(n_vars)` drawn uniformly at random, i.e. a sample from the uniform distribution over $\mathcal{S}_n$, which is the Mallows model with $\theta = 0$. It implements the `pateda` `SeedingMethod` interface, so it can be used both from the wrappers and from `EDAComponents`. Biased initialization — seeding the population with solutions produced by a construction heuristic — is discussed in Section 10.

6.2 Selection Methods

All `pateda` selection operators apply directly, since selection only depends on the fitness values: truncation, proportional, tournament, Boltzmann, and the multi-objective schemes. The plug-and-play wrappers hard-code *truncation selection* with ratio `selection_ratio`, which is the standard choice in the permutation EDA literature (the experimental study of **(author?)** [26] selects the best 10% of a population of size $10n$). Use the component path to employ any other operator.

6.3 Replacement Methods

Likewise, all `pateda` replacement operators are usable. The wrappers implement *elitism* directly: the best permutation found so far is reinserted in place of the worst individual of the new population whenever the new population does not improve on it. Population aggregation — adding the N new solutions to the current population and keeping the best N, the replacement used in **(author?)** [26] — is available as `pateda.replacement.ElitistReplacement` through the component path.

6.4 Stopping Conditions

Inherited from `pateda`: maximum number of generations, maximum number of evaluations, target fitness, and stagnation-based criteria. The wrappers use a fixed number of generations

(n_gen).

6.5 Local Optimization

pateda local search operators that assume a discrete Cartesian representation (bit-flip hill climbing, and the like) are *not* applicable to permutations. Permutation local search must use permutation neighbourhoods — swap, insertion (*jump*), or 2-opt — and can be supplied as a custom LocalOptimizationMethod through EDAComponents. The multi-objective algorithms of Section 8 accept an external perturbation operator directly through the mutation_fn argument (a callable (perm, rng) -> perm) and its application probability mutation_rate, and additionally implement a *shaking* mechanism: after shake_threshold generations without improvement, a subproblem's incumbent is perturbed with shake_strength random insertion moves.

6.6 Repairing Methods

Repair operators, which are essential when a model may generate infeasible solutions, are *unnecessary* in perm_pateda: every sampler returns a valid permutation by construction, either because the model is defined on $\mathcal{S}_n$ and sampled sequentially without replacement, or because the representation is a bijective coding, or because the random-key decoding is a sort. This is the central design advantage of permutation-specific models over the naive use of an integer-vector EDA.

6.7 Mutation and Crossover

perm_pateda does not implement permutation crossover operators (order crossover, PMX, cycle crossover) as first-class components; where a crossover is needed — for example in the candidate-generation step of the Fourier-assisted multi-objective algorithms — it is implemented locally in the corresponding module. Permutation mutation is available through the mutation_fn hook described above.

7 Test Functions and Benchmark Problems

The functions/ package contains seven permutation problems. Each problem is a class whose __call__ returns a value to be *maximized* (Section 3), accompanied by random-instance generators and, where applicable, a loader for the standard benchmark format. Following the organization of perm_mateda, each problem separates instance reading from solution evaluation.

7.1 Classical Permutation Problems

Travelling Salesman Problem (TSP) (functions/tsp.py). An instance is an $n \times n$ matrix of distances between cities. A permutation is a Hamiltonian cycle and its cost is $\sum_{i=1}^{n} D[\sigma(i), \sigma(i \bmod n + 1)]$, including the wrap-around edge; __call__ returns its negation. The implementation validates that the matrix is symmetric. Generators: create_random_tsp(n_cit seed) (random symmetric matrix) and create_tsp_from_coordinates(coords) (Euclidean distances from a set of 2-D points). Reader: parse_tsplib [43].

Permutation Flowshop Scheduling Problem (PFSP) (functions/pfsp.py). An instance is an $n \times m$ matrix of processing times, $P[i, j]$ being the time job i spends on machine j; all jobs traverse the machines in the same order, and a permutation fixes the order of the jobs. Two objectives are supported through the objective argument: "makespan", the completion time of the last job on the last machine, and "flowtime", the sum of the completion times of all jobs. Both are computed from the standard completion-time recurrence $C_{i,j} = \max(C_{i-1,j}, C_{i,j-1}) + P[\sigma(i), j]$, and returned negated. Generator: create_random_pfsp;

reader: `parse_taillard_pfsp` [46] together with `load_taillard_instance`. The distance-based EDA of **(author?)** [6] was designed for this problem.

Linear Ordering Problem (LOP) (`functions/lop.py`). An instance is an $n \times n$ matrix B; the objective is to find the permutation maximizing $\sum_{i<j} B[\sigma(i), \sigma(j)]$, i.e. the sum of the entries above the diagonal of the permuted matrix [21, 35]. It has applications in ranking and aggregation, input–output economics, archaeological seriation and the feedback arc set problem. Generators: `create_random_lop`, `create_tournament_lop` (a random tournament, where the optimum is a ranking), `create_triangular_lop` (an instance with a planted optimum), `create_sparse_lop` (with a prescribed density), plus `feedback_arc_set_to_lop` which converts a directed graph into the equivalent LOP. Reader: `parse_lolib` together with `load_lolib_instance`.

Quadratic Assignment Problem (QAP) (`functions/qap.py`). An instance consists of a flow matrix F between facilities and a distance matrix D between locations [28, 11]; a permutation assigns facilities to locations and its cost is $\sum_{i,j} F[i, j]\, D[\sigma(i), \sigma(j)]$, returned negated by default (`minimize=True`). Generators: `create_random_qap` (with an optional sparse flow matrix), `create_uniform_qap` and `create_grid_qap` (locations on a regular grid, a structured and notoriously hard family). Reader: `parse_qaplib` [4] together with `load_qaplib_instance`.

7.2 Graph Problems under the Permutation Picture

Three classical graph problems are included in the formulation of **(author?)** [37], in which a permutation σ of the vertices together with a cut point k encodes a subset of vertices, and the constraint is expressed through the trace $\mathrm{Tr}(PAP^\top C(k))$ of the permuted adjacency matrix against a truncation matrix $C(k)$. This reformulation turns three subset selection problems into permutation problems, so that any permutation EDA can be applied to them.

Maximum Independent Set (MIS) (`functions/mis.py`). Maximize k subject to $\mathrm{Tr}(PAP^\top C(k)) = 0$, with $C(k)$ the $k \times k$ upper-left block of ones. A permutation is evaluated by scanning it and greedily accumulating the vertices that are not adjacent to any already selected vertex; the fitness is the size of the resulting independent set. Auxiliary methods: `evaluate_independent_set` returns the set itself and `is_valid_independent_set` verifies it. Generators: `create_random_mis(n, edge_probability, seed)` and `create_mis_from_edges(n, edges)`.

Maximum Cut (MaxCut) (`functions/maxcut.py`). Maximize $\frac{1}{2}\mathrm{Tr}(PAP^\top C(k))$ with $C(k)_{ij} = 1$ iff $(i \le k < j)$ or $(j \le k < i)$. A permutation is evaluated by trying all $n - 1$ cut points and returning the largest cut found, so that the encoded solution is the pair (permutation, best cut point). `evaluate_cut` additionally returns the cut point and the two vertex subsets. Generators: `create_random_max_cut`, `create_max_cut_from_edges`.

Minimum Vertex Cover (MVC) (`functions/mvc.py`). Minimize k subject to $\mathrm{Tr}(PAP^\top C(k)) = 0$ with $C(k)_{ij} = 1$ iff $i > k$ and $j > k$, i.e. no edge lies entirely outside the first k vertices. The fitness returned is $-k$, consistent with the maximization convention. Generators: `create_random_mvc`, `create_mvc_from_edges`.

7.3 Benchmark Instance Readers

The module `utils/benchmark_parsers.py` implements readers for the four standard benchmark libraries used in the permutation EDA literature. Each returns plain `numpy` arrays that can be fed directly to the corresponding problem class:

Function	Library	Returns
`parse_lolib(path)`	LOLIB (LOP)	(n, n) weight matrix
`parse_taillard_pfsp(path)`	Taillard (PFSP)	$(n_{\text{jobs}}, n_{\text{machines}})$ times
`parse_qaplib(path)`	QAPLIB (QAP)	(flow, distance) matrices
`parse_tsplib(path)`	TSPLIB (TSP)	(n, n) distance matrix

```python
from perm_pateda import parse_qaplib
from perm_pateda.functions import load_qaplib_instance
from perm_pateda import GMallowsCayleyEDA

flow, dist = parse_qaplib("instances/tai40b.dat")
qap = load_qaplib_instance(flow, dist)

alg = GMallowsCayleyEDA(n_vars=qap.n, fitness_func=qap,
                        pop_size=10 * qap.n, n_gen=500,
                        selection_ratio=0.1, random_seed=111)
stats, _ = alg.run()
print("Best cost:", -stats.best_fitness_overall)
```

8 Multi-Objective Optimization

8.1 Multi-Objective Problem Definition

A multi-objective permutation problem requires the simultaneous optimization of m objectives $f_1(\sigma), \ldots, f_m(\sigma)$ over $\mathcal{S}_n$. Since the objectives usually conflict, the goal is the *Pareto front*: the set of non-dominated permutations, for which no objective can be improved without degrading another. In `perm_pateda` the objectives are supplied as a list of callables, each mapping a 0-indexed permutation to a float, together with a tuple of `minimize` flags (one per objective, all `True` by default), which allows mixing minimization and maximization objectives in a single problem.

8.2 Pareto Dominance and Archive

The module `multiobjective/pareto.py` provides `dominates(a, b, minimize)`, `pareto_front(objectives, minimize)` — returning the indices of the non-dominated rows — and the class `ParetoArchive`, an unbounded external archive. `ParetoArchive.add` inserts a solution only if it is not dominated by an existing member and is not a numerical duplicate of one, and removes every member that the new solution dominates. `get_front()` returns the pair (solutions, objectives).

8.3 Scalarization

The module `multiobjective/scalarization.py` implements the two scalarization functions used by decomposition-based algorithms:

$$g^{ws}(\sigma \mid \mathbf{w}) = \sum_{i=1}^{m} w_i f_i(\sigma), \tag{17}$$

$$g^{te}(\sigma \mid \mathbf{w}, \mathbf{z}^*) = \max_{1 \le i \le m} w_i \, |f_i(\sigma) - z_i^*|, \tag{18}$$

where $\mathbf{z}^*$ is the ideal (reference) point. The Tchebycheff formulation guarantees that every Pareto-optimal solution can be recovered by some weight vector, which the weighted sum does not; it is the default in all the MEDA/D algorithms. Internally these algorithms use *normalized* variants, in which the objectives are rescaled by the current $(\mathbf{z}^*, \mathbf{z}^{\text{nad}})$ box, so that objectives on very different scales (as in bi-objective TSP/QAP combinations) contribute comparably.

8.4 The MEDA/D Family

`perm_pateda` implements the multi-objective decomposition-based EDA framework of **(author?)** [49], which combines the MOEA/D decomposition scheme [50] with permutation probability models. The multi-objective problem is decomposed into N scalar subproblems defined by uniformly spread weight vectors; each subproblem k keeps one incumbent solution, and its neighbourhood $B(k)$ consists of the T subproblems with the closest weight vectors. At each generation, and for each subproblem, a model is estimated from the neighbourhood (or a kernel is centred on the incumbent), one new permutation is sampled from it, evaluated, and used to replace up to `nr` neighbours that it improves under their scalarized objective. The external `ParetoArchive` accumulates the non-dominated solutions found.

The framework is instantiated with nine models, all sharing the same interface:

Class	Model	Module
`MEDA_D_MK`	Kernels of Mallows models, Cayley distance	`meda_d_mk`
`MEDA_D_KENDALL`	Mallows, Kendall's-τ	`meda_d_mk`
`MEDA_D_ULAM`	Mallows, Ulam	`meda_d_mk`
`MEDA_D_GMKENDALL`	Generalized Mallows, Kendall's-τ	`meda_d_mk`
`MEDA_D_GMCAYLEY`	Generalized Mallows, Cayley	`meda_d_mk`
`MEDA_D_PLACKETT_LUCE`	Plackett–Luce	`meda_d_pl`
`MEDA_D_MIXTURE_PLACKETT_LUCE`	Mixture of Plackett–Luce	`meda_d_pl`
`MEDA_D_NHM`	Node histogram model	`meda_d_hm`
`MEDA_D_EHM`	Edge histogram model	`meda_d_hm`

All of them accept the constructor parameters `objectives`, `n`, `n_subproblems` (N), `neighbourhood_size` (T), `nr`, `scalarization`, `shake_threshold`, `shake_strength`, `minimize`, `seed`, and the optional external perturbation operator `mutation_fn` with its probability `mutation_rate` (disabled by default). `MEDA_D_MK` additionally takes the fixed kernel spread `theta`; the histogram variants take `selection_ratio` and `beta_ratio`; the mixture variant takes the number of components. The method `run(n_generations, verbose, initial_population, generation_callback)` returns a dictionary with the keys `pareto_solutions`, `pareto_objectives`, `final_population` and `final_objectives`. The `generation_callback` hook makes it easy to track quality indicators along the run.

```python
import numpy as np
from perm_pateda.multiobjective import MEDA_D_GMCAYLEY
from perm_pateda.functions import create_random_qap,
    create_random_lop

n = 20
qap = create_random_qap(n, seed=1)
lop = create_random_lop(n, seed=2)

# both objectives expressed as minimization
objectives = [lambda p: -qap(p), lambda p: -lop(p)]

alg = MEDA_D_GMCAYLEY(
    objectives=objectives, n=n,
    n_subproblems=50, neighbourhood_size=10, nr=2,
    scalarization="tchebycheff",
    minimize=(True, True), seed=0,
)
result = alg.run(n_generations=200, verbose=True)
print("Pareto front size:", len(result["pareto_solutions"]))
```

8.5 Relation to the `pateda` Multi-Objective Module

`pateda` addresses multi-objective problems mainly by modifying the selection scheme (non-dominated sorting, crowding distance, indicator-based selection) and provides quality indicators, an external archive and a MOEA/D implementation; see the corresponding section of the `pateda` user guide. Those components can be combined with the permutation learners and samplers through `EDAComponents` whenever a Pareto-selection-based rather than a decomposition-based algorithm is desired. The MEDA/D classes described above are self-contained implementations that do not go through the `pateda` executor, because the model is re-estimated per subproblem rather than once per generation.

9 Model Inspection and Statistical Analysis

9.1 Inspecting the Learned Models

Because every learner returns a plain dictionary, the model is directly inspectable and can be logged at every generation. The most informative quantities are:

Key	Meaning
`consensus`	Central permutation σ_0 of a Mallows/GM model; its evolution shows which orderings the search converges to
`theta`	Spread parameter; the transition from small to large θ marks the exploration-to-exploitation switch of the run
`thetas`	The $(n-1)$ per-position spreads of a GM model; positions with large θ_j have already converged
`ehm_matrix`, `nhm_matrix`	Edge/node frequency matrices; their entropy is a direct diversity measure
`weights`	Plackett–Luce item weights, i.e. an estimated ranking
`dsm`	Doubly stochastic matrix; its distance to a permutation matrix measures convergence
`marginals`, `conditionals`	Tables of the models over bijective codings, in the same format used by `pateda`

Tracking θ across generations reproduces the two-phase behaviour reported by **(author?)** [26]: in the first generations θ takes very low values, denoting a large spread among the solutions in the population (exploration), and later it increases rapidly, indicating convergence (exploitation). At that point the probability of sampling a solution far from the consensus becomes extremely low, so practitioners may wish to cap the maximum θ (parameter `upper_theta` of the learners) to delay stagnation.

Since the models over bijective codings (Section 5.12) are ordinary discrete probabilistic graphical models, the whole `knowledge_extraction` module of `pateda` — dependency measures, network measures, structure visualization — applies to them without modification.

9.2 Statistical Comparison Utilities

The module `utils/stats_utils.py` implements the comparison protocol used in the permutation EDA literature [8, 15]. Its input is a `pandas DataFrame` whose rows are problem instances and whose columns are algorithms:

Function	Description
`summary_table(df)`	Mean ± standard deviation, best value and average rank per algorithm
`friedman_test(df)`	Friedman omnibus test across algorithms; returns the statistic, the p-value and the average ranks
`wilcoxon_pairwise(df)`	Matrix of pairwise Wilcoxon signed-rank p-values
`critical_difference_plot(df, alpha, filepath)`	Critical-difference diagram (Nemenyi) saved to file

10 Injecting A-Priori Knowledge

`pateda` offers several mechanisms to inject a-priori problem-structure information into an EDA: fixed factorizations, interaction matrices constraining the structures a learner may consider, biased initialization, and constrained search. Their general description is given in the `pateda` user guide. In the permutation setting they specialize as follows.

Biased initialization Any set of permutations — for instance produced by a construction heuristic such as NEH for the PFSP or nearest-neighbour for the TSP — can be passed as the initial population. The multi-objective algorithms accept it directly through the `initial_population` argument of `run()`; for the single-objective wrappers, replace the seeding component through `EDAComponents`.

Structural priors The tree models over bijective codings accept a `root` argument that fixes the orientation of the dependency tree; a problem-informed choice of root (e.g. the position known to be most constrained) acts as a mild structural prior. Restricting the set of candidate edges through an interaction matrix, as `pateda` does for Tree-EDA_r, transfers directly to `LearnTree`.

Model priors The smoothing parameters play the role of a prior over the model: `beta_ratio` in the histogram models, `alpha` in the DSM models, and `laplace_smoothing` in the coding-based models. Larger values keep more of the uniform distribution in the mixture and thus slow down convergence. In the Mallows kernels, the schedule $(E_0, E_{\max}, \gamma)$ is an explicit, problem-independent prior on the exploration/exploitation trade-off.

Choice of distance and model The strongest form of a-priori knowledge in a permutation EDA is the choice of the distance or model family itself, since it determines which discrepancies between permutations the search considers small. Section 5 summarizes the recommended correspondences.

11 Relation to `perm_mateda` and Coverage

`perm_pateda` takes as its reference point the `perm_mateda` Matlab toolbox [26], which extended MATEDA-2.0 [44] with the Mallows and generalized Mallows models under the Kendall's-τ, Cayley and Ulam distances, three central-permutation estimators, the matched samplers, and four permutation problems. Table 3 summarizes the correspondence.

Two differences with respect to the original toolbox deserve mention. First, the Ulam sampler is a Metropolis–Hastings chain rather than the exact "distances sampler" based on random standard Young tableaux; the exact sampler remains a planned addition. Second, the estimation of θ for the Ulam distance uses the exact count of permutations at each distance only for $n \leq 8$, and a Gaussian approximation of that count for larger n.

Component	`perm_mateda`	`perm_pateda`
MEDA-Kendall	yes	`MallowsKendallEDA`
MEDA-Cayley	yes	`MallowsCayleyEDA`
MEDA-Ulam	yes	`MallowsUlamEDA` (MCMC sampler)
GMEDA-Kendall	yes	`GMallowsKendallEDA`
GMEDA-Cayley	yes	`GMallowsCayleyEDA`
Borda consensus	yes	`find_consensus_borda`
SetMedian consensus	yes	`find_consensus_median`
BestPermutation consensus	yes	`find_consensus_best`
TSP, PFSP, LOP, QAP	yes	`functions/`
TSPLIB / Taillard / LOLIB / QAPLIB readers	yes	`utils/benchmark_parsers.py`
Kernels of Mallows models (Hamming)	—	`HammingKMMEDA`
Edge / node histogram models	—	`EHMEDA`, `NHMEDA`
Plackett–Luce and mixtures	—	`PlackettLuceEDA`, `PlackettLuceMixtureE`
Doubly stochastic matrix models	—	`DSMPSEDA`, `DSMASEDA`
Bijective codings + UMDA/tree/Markov	—	`representations/`, six wrappers
Random-key EDAs	—	`RKGaussianUMDAEDA`, `RKGaussianFullEDA`, `RKCopul`
Graph problems (MIS, MaxCut, MVC)	—	`functions/mis.py`, `maxcut.py`, `mvc.py`
Multi-objective MEDA/D (nine models)	—	`multiobjective/`
Statistical comparison utilities	—	`utils/stats_utils.py`

Table 3: Coverage of `perm_mateda` by `perm_pateda`, and functionality added beyond it.

12 Conclusions

`perm_pateda` is a Python library of estimation of distribution algorithms for permutation-based combinatorial optimization. It is built as a specialization of `pateda`: the EDA executor, the component architecture, selection, replacement, statistics and stopping conditions are inherited unchanged, and what the library contributes is the machinery that is specific to the symmetric group — distances and their decompositions, central-permutation estimators, seventeen learning/sampling pairs covering distance-based, multistage, histogram, matrix-valued, coding-based and continuous-relaxation models, seven benchmark problems with their instance readers, and a decomposition-based multi-objective framework instantiated with nine models.

Beyond reproducing the five distance-based EDAs of the `perm_mateda` toolbox in Python, the library incorporates a set of developments that go beyond it:

- kernels of Mallows models under the Hamming distance, with an explicit exploration/exploitation schedule;
- multistage models (Plackett–Luce) and their mixtures, learned with spectral EM and sampled with the Gumbel-max trick;
- doubly stochastic matrix models with two sampling strategies;
- bijective codings of $\mathcal{S}_n$ that make the full catalogue of discrete `pateda` models applicable to permutations without repair;
- random-key EDAs that bring the continuous models — including vine copulas — to bear on permutation problems;
- graph problems reformulated under the permutation picture;
- a multi-objective decomposition framework covering nine permutation models;
- instance readers and statistical comparison tools that make experimental studies reproducible.

Ongoing work includes the exact Ulam sampler based on random standard Young tableaux, permutation-specific local search operators as first-class components, the integration of the Fourier-decomposition models currently kept in the companion `permutation_fourier` project, and a validation of the parameter estimates and sampled-distance histograms against the reference outputs of the Matlab `perm_mateda` toolbox.

References

[1] E. Arza, J. Ceberio, E. Irurozki, and A. Pérez. Kernels of Mallows models under the Hamming distance for solving the quadratic assignment problem. *Swarm and Evolutionary Computation*, 59:100740, 2020.

[2] M. Ayodele, J. McCall, and O. Regnier-Coudert. RK-EDA: A novel random key based estimation of distribution algorithm. In *Parallel Problem Solving from Nature – PPSN XIV*, volume 9921 of *Lecture Notes in Computer Science*, pages 849–858. Springer, 2016.

[3] J. C. Bean. Genetic algorithms and random keys for sequencing and optimization. *ORSA Journal on Computing*, 6(2):154–160, 1994.

[4] R. E. Burkard, S. E. Karisch, and F. Rendl. QAPLIB – a quadratic assignment problem library. *Journal of Global Optimization*, 10(4):391–403, 1997.

[5] J. Ceberio, E. Irurozki, A. Mendiburu, and J. A. Lozano. A review on estimation of distribution algorithms in permutation-based combinatorial optimization problems. *Progress in Artificial Intelligence*, 1(1):103–117, 2012.

[6] J. Ceberio, E. Irurozki, A. Mendiburu, and J. A. Lozano. A distance-based ranking model estimation of distribution algorithm for the flowshop scheduling problem. *IEEE Transactions on Evolutionary Computation*, 18(2):286–300, 2014.

[7] J. Ceberio, E. Irurozki, A. Mendiburu, and J. A. Lozano. Extending distance-based ranking models in estimation of distribution algorithms. In *2014 IEEE Congress on Evolutionary Computation (CEC)*, pages 2459–2466. IEEE, 2014.

[8] J. Ceberio, E. Irurozki, A. Mendiburu, and J. A. Lozano. A review of distances for the Mallows and generalized Mallows estimation of distribution algorithms. *Computational Optimization and Applications*, 62(2):545–564, 2015.

[9] J. Ceberio, A. Mendiburu, and J. A. Lozano. Introducing the Mallows model on estimation of distribution algorithms. In *Neural Information Processing (ICONIP 2011)*, volume 7063 of *Lecture Notes in Computer Science*, pages 461–470. Springer, 2011.

[10] J. Ceberio, A. Mendiburu, and J. A. Lozano. The Plackett-Luce ranking model on permutation-based optimization problems. In *2013 IEEE Congress on Evolutionary Computation (CEC)*, pages 494–501. IEEE, 2013.

[11] E. Çela. *The Quadratic Assignment Problem: Theory and Algorithms*. Kluwer Academic Publishers, Dordrecht, 1998.

[12] F. Chicano, D. Whitley, G. Ochoa, and R. Tinós. Fourier transform-based surrogates for permutation problems. In *Proceedings of the Genetic and Evolutionary Computation Conference (GECCO 2023)*, pages 275–283. ACM, 2023.

[13] C. K. Chow and C. N. Liu. Approximating discrete probability distributions with dependence trees. *IEEE Transactions on Information Theory*, 14(3):462–467, 1968.

[14] J.-C. de Borda. Mémoire sur les élections au scrutin. *Histoire de l'Académie Royale des Sciences*, 1781.

[15] J. Demšar. Statistical comparisons of classifiers over multiple data sets. *Journal of Machine Learning Research*, 7:1–30, 2006.

[16] P. Diaconis. *Group Representations in Probability and Statistics*, volume 11 of *Institute of Mathematical Statistics Lecture Notes—Monograph Series*. Institute of Mathematical Statistics, Hayward, CA, 1988.

[17] R. Durstenfeld. Algorithm 235: Random permutation. *Communications of the ACM*, 7(7):420, 1964.

[18] R. A. Fisher and F. Yates. *Statistical Tables for Biological, Agricultural and Medical Research*. Oliver and Boyd, London, 1938.

[19] M. A. Fligner and J. S. Verducci. Distance based ranking models. *Journal of the Royal Statistical Society. Series B (Methodological)*, 48(3):359–369, 1986.

[20] M. A. Fligner and J. S. Verducci. Multistage ranking models. *Journal of the American Statistical Association*, 83(403):892–901, 1988.

[21] M. Grötschel, M. Jünger, and G. Reinelt. A cutting plane algorithm for the linear ordering problem. *Operations Research*, 32(6):1195–1220, 1984.

[22] D. R. Hunter. MM algorithms for generalized Bradley-Terry models. *The Annals of Statistics*, 32(1):384–406, 2004.

[23] E. Irurozki. *Sampling and Learning Distance-based Probability Models for Permutation Spaces*. PhD thesis, Faculty of Computer Science, University of the Basque Country UPV/EHU, 2014.

[24] E. Irurozki, B. Calvo, and J. A. Lozano. Mallows model under the Ulam distance: A feasible combinatorial approach. In *NIPS Workshop on Analysis of Rank Data*, 2014.

[25] E. Irurozki, B. Calvo, and J. A. Lozano. Sampling and learning Mallows and generalized Mallows models under the Cayley distance. *Methodology and Computing in Applied Probability*, 20(1):1–35, 2018.

[26] E. Irurozki, J. Ceberio, J. Santamaría, R. Santana, and A. Mendiburu. Algorithm 989: perm_mateda: A Matlab toolbox of estimation of distribution algorithms for permutation-based combinatorial optimization problems. *ACM Transactions on Mathematical Software*, 44(4):47:1–47:13, 2018.

[27] R. Kondor, A. Howard, and T. Jebara. Multi-object tracking with representations of the symmetric group. In *Proceedings of the 11th International Conference on Artificial Intelligence and Statistics (AISTATS)*, pages 211–218, 2007.

[28] T. C. Koopmans and M. J. Beckmann. Assignment problems and the location of economic activities. *Econometrica*, 25(1):53–76, 1957.

[29] P. Larrañaga and J. A. Lozano, editors. *Estimation of Distribution Algorithms. A New Tool for Evolutionary Computation*. Kluwer Academic Publishers, Boston/Dordrecht/London, 2002.

[30] D. H. Lehmer. Teaching combinatorial tricks to a computer. *Proceedings of Symposia in Applied Mathematics*, 10:179–193, 1960.

[31] J. A. Lozano, P. Larrañaga, I. Inza, and E. Bengoetxea, editors. *Towards a New Evolutionary Computation: Advances on Estimation of Distribution Algorithms*. Springer, 2006.

[32] R. D. Luce. *Individual Choice Behavior: A Theoretical Analysis*. Wiley, New York, 1959.

[33] C. L. Mallows. Non-null ranking models. I. *Biometrika*, 44(1–2):114–130, 1957.

[34] B. Mandhani and M. Meilă. Tractable search for learning exponential models of rankings. *Journal of Machine Learning Research (Proceedings of AISTATS)*, 5:392–399, 2009.

[35] R. Martí and G. Reinelt. *The Linear Ordering Problem: Exact and Heuristic Methods in Combinatorial Optimization*, volume 175 of *Applied Mathematical Sciences*. Springer, 2011.

[36] L. Maystre and M. Grossglauser. Fast and accurate inference of Plackett–Luce models. In *Advances in Neural Information Processing Systems 28 (NeurIPS)*, pages 172–180, 2015.

[37] Y. Min. Permutation picture of graph combinatorial optimization problems. *arXiv preprint arXiv:2410.17111*, 2024.

[38] H. Mühlenbein and G. Paaß. From recombination of genes to the estimation of distributions I. Binary parameters. In H.-M. Voigt, W. Ebeling, I. Rechenberg, and H.-P. Schwefel, editors, *Parallel Problem Solving from Nature - PPSN IV*, volume 1141 of *Lectures Notes in Computer Science*, pages 178–187, Berlin, 1996. Springer.

[39] D. Nguyen and A. Y. Zhang. Efficient and accurate learning of mixtures of Plackett–Luce models. In *Proceedings of the 37th AAAI Conference on Artificial Intelligence (AAAI-23)*, pages 9294–9301, 2023.

[40] M. Pelikan, K. Sastry, and E. Cantú-Paz, editors. *Scalable Optimization via Probabilistic Modeling: From Algorithms to Applications*. Studies in Computational Intelligence. Springer, 2006.

[41] R. L. Plackett. The analysis of permutations. *Journal of the Royal Statistical Society. Series C (Applied Statistics)*, 24(2):193–202, 1975.

[42] O. Regnier-Coudert and J. McCall. Factoradic representation for permutation optimisation. In *Parallel Problem Solving from Nature – PPSN XIII*, volume 8672 of *Lecture Notes in Computer Science*, pages 332–341. Springer, 2014.

[43] G. Reinelt. TSPLIB – a traveling salesman problem library. *ORSA Journal on Computing*, 3(4):376–384, 1991.

[44] R. Santana, P. Larrañaga, and J. A. Lozano. Research topics on discrete estimation of distribution algorithms. *Memetic Computing*, 1(1):35–54, 2009.

[45] V. Santucci and J. Ceberio. Doubly stochastic matrix models for estimation of distribution algorithms. *arXiv preprint arXiv:2304.02458*, 2023.

[46] É. Taillard. Benchmarks for basic scheduling problems. *European Journal of Operational Research*, 64(2):278–285, 1993.

[47] S. Tsutsui. Probabilistic model-building genetic algorithms in permutation representation domain using edge histogram. In *Parallel Problem Solving from Nature – PPSN VII*, volume 2439 of *Lecture Notes in Computer Science*, pages 224–233. Springer, 2002.

[48] S. Tsutsui, M. Pelikan, and D. E. Goldberg. Node histogram vs. edge histogram: A comparison of probabilistic model-building genetic algorithms in permutation domains. In *2006 IEEE Congress on Evolutionary Computation (CEC)*, pages 1939–1946. IEEE, 2006.

[49] M. Zangari, A. Mendiburu, R. Santana, and A. Pozo. Multiobjective decomposition-based Mallows models estimation of distribution algorithm. A case of study for permutation flowshop scheduling problem. *Information Sciences*, 397–398:137–154, 2017.

[50] Q. Zhang and H. Li. MOEA/D: A multiobjective evolutionary algorithm based on decomposition. *IEEE Transactions on Evolutionary Computation*, 11(6):712–731, 2007.